

PREDICTION TASK  What is the type of task? Which entity are predictions made on? What are the possible outcomes to predict? When are outcomes observed? TASK TYPE <i>Sequence prediction (i.e., predict the next access key in a sequence).</i> PREDICTION ENTITIES <i>Data keys (i.e., unique identifiers for data items).</i> PREDICTION OUTCOMES <i>Logits representing the likelihood of each key of being accessed at each future step.</i> OUTCOMES OBSERVATION <i>At the end of the predicted future time horizon.</i>	DECISIONS  How are predictions turned into actionable recommendations or decisions for the end-user? (Mention parameters of the process / application for this.) FROM PREDICTIONS TO DECISIONS <i>Model logits and variances (over a predefined time horizon) are combined into normalized eviction scores via confidence-weighted logic (for a given confidence level). These scores are used to rank cache keys, where higher scores indicate lower eviction priority, and keys with the lowest scores—excluding protected ones—are selected for eviction.</i>	VALUE PROPOSITION  Who is the end beneficiary, and what specific pain points are addressed? How will the ML solution integrate with their workflow, and through which user interfaces? END BENEFICIARY <i>Operations/platform teams integrating the solution into a specific system (direct) and its corresponding end-users (indirect).</i> PAIN POINTS ADDRESSED 1) Ineffective cache eviction strategies: - Rigid, rule-based policies. - Poor suitability for real-world, non-stationary data access patterns. - No use of predictive or probabilistic information. 2) Inefficient resource utilization: - Low cache hit rate. - Suboptimal use of fast cache memory. 3) High operational and infrastructure costs: - Increased load on backing services. - Higher infrastructure and operational expenses. 4) Degraded service performance and User Experience (UX): - Increased end-user latency. - Lower quality of service. - Negative impact on UX. - Risk of violating Service Level Objectives (SLOs). ML SOLUTION IN THE WORKFLOW <i>The solution is integrated via a RESTful API within a microservice-based architecture. The cache management layer provides recent access data information, current cache state, and optional configuration settings, and receives the key(s) to evict when inserting new item(s) into a full cache. The eviction decision is fully programmatic; no user interface is required.</i>	DATA COLLECTION  How is the initial set of entities and outcomes sourced (e.g., database extracts, API pulls, manual labeling)? What strategies are in place to update data continuously while controlling cost and maintaining freshness? INITIAL DATA SOURCING <i>The initial dataset is sourced via programmatic simulation of key access sequences.</i> CONTINUOUS DATA UPDATES <i>Real-world access events, already preprocessed for API use, are logged as structured events and appended to the dataset for continuous updates. Only recent access events within a fixed lookback window are used, ensuring fresh data and limiting processing costs.</i>	DATA SOURCES  Where can we get data on entities and observed outcomes? (Mention internal and external database tables or API methods.) DATA SOURCES - Synthetic generation: Programmatic simulation environment used to construct the initial dataset of access sequences. - Internal logging system: Centralized repository storing real-world, structured access events captured via API calls.
IMPACT SIMULATION  What are the cost/gain values for (in)correct decisions? Which data is used to simulate pre-deployment impact? What are the criteria for deployment? Are there fairness constraints? COST/GAIN OF DECISIONS <i>Correct eviction decisions increase cache hit rate and reduce average system response latency, while incorrect ones increase misses and latency.</i> PRE-DEPLOYMENT IMPACT DATA <i>Pre-deployment impact is simulated offline using a hold-out of historical or collected key access sequences replayed through a cache simulator with the ML eviction API, comparing hit rate, miss rate, and average latency against LRU, LFU, and Belady's MIN (optimal benchmark).</i> CRITERIA FOR DEPLOYMENT <i>Deployment requires passing post-training behavioral tests, outperforming baselines without significant train-test degradation, and meeting inference latency and throughput targets. Offline cache impact simulation can provide additional validation. The model is first staged with canary testing; a fraction of traffic is used to evaluate business metrics, guiding manual promotion to production.</i> FAIRNESS CONSTRAINTS <i>No fairness constraints; the model only predicts cache key eviction.</i>	MAKING PREDICTIONS  Are predictions made in batch or in real time? How frequently? How much time is available for this (including featurization and decisions)? Which computational resources are used? PREDICTION TYPE AND FREQUENCY <i>Real-time and event-driven, triggered whenever the cache requires an eviction decision.</i> LATENCY CONSTRAINTS <i>The end-to-end latency is bounded to 3000 ms and is fully asynchronous, occurring outside the user request critical path.</i> COMPUTATIONAL RESOURCES <i>Server-side logic and inference using CPU-based resources within a scalable microservice architecture.</i> MONITORING Which metrics and KPIs are used to track the ML solution's impact once deployed, both for end-users and for the business? How often should they be reviewed?	METRICS AND KPIS <i>Model performance is tracked via accuracy, precision, recall, and F1 (macro and weighted) against a baseline, with train-test degradation checks. API and infrastructure metrics—such as request/response sizes, throughput, p95 latency (end-to-end and per-microservice), success rate, and CPU/memory usage—are monitored for operational health. KPIs, including cache hit and miss rates, should be tracked externally by the platform/operations team, as these metrics are not directly accessible.</i> REVIEW FREQUENCY <i>On alerts or weekly; routine dashboards and reports available as needed.</i>	BUILDING MODELS  How many models are needed in production? When should they be updated? How much time is available for this (including featurization and analysis)? Which computation resources are used? MODELS IN PRODUCTION <i>One LSTM model runs in production; a second may run in staging for canary testing. Full-precision, unpruned copies are retained for future retraining.</i> MODEL UPDATE SCHEDULE <i>Automatically on a weekly basis or manually when performance, drift indicators, or business metrics degrade, provided sufficient new data exists. The full process completes within 120 minutes.</i> COMPUTATIONAL RESOURCES <i>Updating is performed offline using self-hosted, containerized CPU-based resources.</i>	FEATURES  What representations are used for entities at prediction time? What aggregations or transformations are applied to raw data sources? ENTITY REPRESENTATIONS <i>Keys are represented by three types of features:</i> - Temporal: Sine and cosine of timestamp (cyclical pattern). - Local: Normalized frequency and recency within lookback window. - Identity: Raw key mapped to a learnable embedding. DATA TRANSFORMATIONS AND AGGREGATIONS <i>Raw data (timestamps and keys) is processed as follows:</i> 1) Remove rows with missing values. 2) Encode timestamps trigonometrically, transforming them into sine and cosine components. 3) Compute local frequency, representing the normalized count of key occurrences within the lookback window. 4) Compute local recency, representing the normalized and reversed distance from key's last occurrences within the lookback window. <i>Preprocessed data is then structured into input sequences of fixed length, providing the model with temporal context for predictions.</i>